

Resumen para Exposicion - Proyecto Final

Integrantes

- Natalia Martinez Villegas
- Emanuel Correa Jimenez
- Oscar Marin Zamora

Idea general del proyecto

El proyecto final consistio en implementar una arquitectura funcional en Oracle Cloud Infrastructure usando Terraform. La solucion tiene una aplicacion web publica conectada a una base de datos privada.

La idea principal fue separar la capa de aplicacion de la capa de datos. El usuario entra desde Internet a una pagina web hecha con Python Flask. Esa aplicacion esta en una maquina virtual publica. Cuando se llena el formulario HTML, Flask recibe el dato y lo guarda en MariaDB, que esta instalada en una segunda maquina virtual privada.

La base de datos no tiene IP publica, por lo que no se puede acceder directamente desde Internet. Esto permite publicar la aplicacion sin exponer la informacion almacenada.

Infraestructura creada

La infraestructura se creo en Oracle Cloud Infrastructure mediante Terraform.

Se creo una VCN llamada `lab4-vcn` con el bloque:

```
10.0.0.0/16
```

Dentro de esa VCN se crearon dos subredes:

```
Subred publica: 10.0.1.0/24
Subred privada: 10.0.2.0/24
```

En la subred publica se coloco la VM de aplicacion:

```
Nombre: lab4-oci-vm
IP publica: 129.80.190.44
IP privada: 10.0.1.176
Sistema operativo: Oracle Linux 9
Funcion: ejecutar Flask y recibir trafico web
```

En la subred privada se coloco la VM de base de datos:

```
Nombre: proyecto-final-db-vm
IP privada: 10.0.2.45
IP publica: no tiene
Sistema operativo: Oracle Linux 9
Funcion: alojar MariaDB
```

Tambien se crearon estos componentes:

```
Internet Gateway: permite que la VM publica se comuniquen con Internet.
NAT Gateway: permite salida controlada desde la VM privada sin exponerla.
Route Tables: definen por donde viaja el trafico.
Security Lists: controlan que puertos y origenes se permiten.
```

Como se conecto todo

El flujo de comunicacion del proyecto es:

```
Usuario
|
v
Internet
|
v
VM publica: lab4-oci-vm
|
v
Aplicacion Flask
|
v
Red privada OCI
|
v
VM privada: proyecto-final-db-vm
|
v
MariaDB
```

La VM publica recibe las solicitudes web desde el navegador. La aplicacion Flask procesa el formulario y se conecta internamente a MariaDB usando:

```
DB_HOST=10.0.2.45
DB_PORT=3306
```

La conexión entre Flask y MariaDB ocurre por la red privada de OCI, no por Internet.

Por que se hizo de esta forma

La razón principal fue seguridad. Si la base de datos tuviera IP pública, estaría más expuesta. Con esta arquitectura:

```
La aplicación si es pública.
La base de datos queda privada.
Solo Flask puede comunicarse con MariaDB.
El usuario nunca accede directamente a la base de datos.
```

Esto representa una arquitectura de dos capas:

```
Capa web pública
Capa de datos privada
```

También se usó Terraform porque permite crear infraestructura como código. Eso significa que la red, las subredes, las máquinas virtuales, las rutas y las reglas de seguridad quedan documentadas en archivos `.tf`. Además, se pueden validar antes de aplicar cambios y se puede comprobar si la nube coincide con el código.

Que buscábamos demostrar

El proyecto buscaba demostrar que podíamos:

1. Crear infraestructura real en la nube usando Terraform.
2. Separar una aplicación web de una base de datos.
3. Exponer solamente la aplicación al público.
4. Proteger la base de datos en una red privada.
5. Validar conectividad entre una VM pública y una VM privada.
6. Desplegar una aplicación Flask funcional.
7. Guardar datos desde un formulario HTML en MariaDB.

Terraform

Los archivos principales de Terraform fueron:

```
provider.tf: configuracion del proveedor OCI.
variables.tf: variables como tenancy, usuario, region y compartment.
```

```
network.tf: VCN, subredes, gateways, rutas y security lists.  
compute.tf: VM publica.  
compute-db.tf: VM privada para base de datos.  
outputs.tf: salidas como IP publica, IP privada y nombres de recursos.
```

Comandos importantes usados durante el proyecto:

```
terraform init  
terraform validate  
terraform plan  
terraform apply  
terraform output  
terraform state list
```

Una validacion clave fue:

```
No changes. Your infrastructure matches the configuration.
```

Ese mensaje significa que Terraform confirmo que lo creado en OCI coincidia con el codigo local.

Seguridad

En la subred publica se permitieron estos puertos:

```
SSH: puerto 22  
HTTP: puerto 80  
HTTPS: puerto 443
```

En la subred privada se permitio:

```
SSH interno desde la subred publica.  
MariaDB por el puerto 3306 solo desde 10.0.1.0/24.  
ICMP interno para pruebas de conectividad.
```

La VM privada no tiene IP publica. Este es uno de los puntos mas importantes de la exposicion porque demuestra que la base de datos queda aislada de Internet.

Python utilizado

La aplicacion fue hecha con Python usando Flask.

Flask se encargo de:

```
Mostrar la pagina web.  
Recibir el formulario.  
Procesar el dato enviado por POST.  
Conectarse a MariaDB usando PyMySQL.  
Insertar datos en la tabla usuarios.  
Consultar los registros guardados.  
Mostrar el listado actualizado en pantalla.
```

Tambien se uso PyMySQL como conector entre Python y MariaDB.

La logica general de la aplicacion fue:

```
Usuario llena formulario  
Flask recibe el nombre  
Flask abre conexion a MariaDB en 10.0.2.45  
Inserta el registro en la tabla usuarios  
Consulta los usuarios existentes  
Devuelve la pagina HTML actualizada
```

HTML del formulario

El HTML fue el formulario visible en la aplicacion web. La pantalla mostraba la seccion **Usuarios**, un campo para ingresar el nombre y un boton **Guardar**.

La funcion del formulario era capturar datos del usuario y enviarlos al backend Flask mediante una solicitud **POST**.

Elementos principales:

```
Campo: Nombre  
Boton: Guardar  
Tabla/listado: usuarios registrados
```

Cuando se guardaba un nombre, la aplicacion actualizaba la lista con los datos recuperados desde MariaDB.

Base de datos

En MariaDB se creo:

```
Base de datos: proyecto_final  
Usuario de aplicacion: flaskuser
```

Tabla: usuarios

La tabla **usuarios** almacena los datos ingresados desde el formulario. La persistencia se validó con:

```
SELECT * FROM usuarios;
```

También se observó un registro como:

Grupo Nati Ema y Oscar

Ese registro sirvió como evidencia de que el formulario sí guardaba y leía datos desde la base de datos.

Pruebas realizadas

Las pruebas principales fueron:

```
SSH a la VM publica.  
Ping desde la VM publica hacia la VM privada.  
SSH interno hacia la VM privada.  
Acceso web a http://129.80.190.44/  
Formulario visible en navegador.  
Inserción de datos desde el formulario.  
Consulta de registros en MariaDB.  
Terraform plan sin cambios.
```

Frase corta para la exposición

Implementamos una arquitectura de dos capas en OCI. La capa pública contiene una VM con Flask que recibe las solicitudes web, y la capa privada contiene una VM con MariaDB sin IP pública. La aplicación se conecta a la base de datos por la red interna usando el puerto **3306**. Todo esto fue creado y validado con Terraform, lo que nos permitió administrar la infraestructura como código y demostrar una solución funcional y más segura.

Reparto sugerido para exponer

Oscar puede explicar la infraestructura y Terraform:

```
VCN  
Subred publica  
Subred privada  
Gateways  
Rutas
```

Security Lists
Validaciones de Terraform

Natalia puede explicar el documento, evidencias, arquitectura general y pruebas:

Objetivo del proyecto
Diagrama general
Evidencias
Pruebas realizadas
Resultados
Conclusiones

Emanuel puede explicar la parte funcional:

MariaDB
Flask
PyMySQL
Formulario HTML
Insercion de datos
Consulta de registros

Idea central para recordar

No es solamente una pagina web. Lo importante es que la pagina web publica se conecta de forma segura a una base de datos privada en la nube.

La separacion entre la VM publica y la VM privada es el punto principal del proyecto.